

LAPORAN TUGAS BESAR

IF2224 Teori Bahasa Formal dan Otomata

“Milestone 4 - Intermediate Code Generation and Interpreter”



Dipersiapkan oleh:

Nama lengkap	NIM
Fauzan Mohamad Abdul Ghani	13524113
Safira Berlianti	13524128
Eduard Daniel Ariajaya	13524129
Muhammad Daffa Arrizki Yanma	13524133
Reysha Syafitri MR	13524137

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Contents

1	Landasan Teori	2
1.1	Intermediate Code (Kode Antara)	2
1.2	Arsitektur Stack Machine	2
1.3	Instruction Set (Set Instruksi)	3
1.4	Kode Operasi (OPR)	4
1.5	Mekanisme Pemanggilan Subprogram	5
2	Perancangan dan Implementasi	5
2.1	Perancangan Program	5
2.2	Konvensi Tambahan di Luar Spesifikasi Dasar	7
2.3	Hasil Implementasi	9
3	Pengujian	13
4	Kesimpulan dan Saran	37
4.1	Kesimpulan	37
4.2	Saran	38
5	Lampiran	38
6	Referensi	40

1 Landasan Teori

1.1 Intermediate Code (Kode Antara)

Intermediate Code (IC) atau kode antara adalah representasi perantara dari program sumber yang dihasilkan oleh *compiler* setelah tahap analisis (leksikal, sintaksis, dan semantik) selesai dilakukan. Kode antara ini berada di antara representasi tingkat tinggi (*source code*) dan representasi tingkat rendah (*machine code*) sehingga bersifat independen terhadap arsitektur mesin target tertentu.

- **Tujuan:** Tujuan utama dari pembuatan kode antara adalah untuk memisahkan *front-end* kompilator (analisis bahasa sumber) dari *back-end* (pembangkitan kode mesin). Dengan pemisahan ini, satu *front-end* dapat dipasangkan dengan banyak *back-end* untuk menghasilkan kode mesin berbagai platform, atau sebaliknya. Selain itu, kode antara mempermudah proses optimasi karena transformasi dapat dilakukan pada tingkat abstraksi menengah tanpa harus memikirkan detail arsitektur.
- **Bentuk Representasi:** Terdapat beberapa bentuk representasi kode antara yang umum digunakan, antara lain:
 - ***Three-Address Code* (TAC):** Setiap instruksi memiliki paling banyak tiga operand (dua sumber dan satu tujuan). Contoh: $t1 = a + b$.
 - ***Stack Machine Code:*** Instruksi beroperasi pada sebuah *stack* tanpa membutuhkan register eksplisit. Operand di-*push* ke *stack* dan operator mengambil operand dari puncak *stack*.
 - ***Directed Acyclic Graph* (DAG):** Representasi graf yang mengeliminasi sub-ekspresi yang berulang (*common subexpressions*).
- **Kaitan dengan Decorated AST:** Pada arsitektur kompilator Arion, kode antara dihasilkan dari *Decorated Abstract Syntax Tree* (Decorated AST) yang telah diperkaya dengan informasi tipe data, indeks *symbol table*, dan level leksikal oleh *Semantic Analyzer* pada Milestone 3. Informasi dekorasi ini digunakan oleh *Intermediate Code Generator* untuk menentukan ukuran memori, jenis operasi, dan alamat variabel secara tepat.

1.2 Arsitektur Stack Machine

Stack Machine adalah model komputasi di mana operasi aritmatika dan logika dilakukan menggunakan sebuah *operand stack* alih-alih register bernama. Model ini dipilih karena kesederhanaannya dalam memetakan ekspresi aritmatika dari pohon sintaks.

- **Prinsip Kerja:** Instruksi *stack machine* umumnya mengikuti pola berikut: operand di-*push* ke puncak *stack* menggunakan instruksi LIT atau LOD; operator (misalnya

OPR) mengambil (*pop*) satu atau dua operand dari puncak *stack*, melakukan operasi, lalu meletakkan (*push*) hasilnya kembali ke puncak *stack*; hasil akhir kemudian disimpan ke memori menggunakan instruksi **STO** atau **STI**.

- **Komponen Utama:**

- ***Program Counter (PC)***: Menyimpan alamat instruksi yang sedang dieksekusi. Nilainya bertambah secara sekuensial, kecuali terdapat instruksi *jump* (JMP, JPC).
- ***Operand Stack***: Struktur data LIFO (*Last-In First-Out*) yang menyimpan nilai sementara selama evaluasi ekspresi.
- ***Global Memory***: Area memori linier yang menyimpan variabel global program.
- ***Activation Frame***: Struktur data yang dialokasikan saat pemanggilan *procedure* atau *function*. Setiap *frame* menyimpan variabel lokal, parameter, alamat kembali (*return address*), serta (untuk *function*) slot untuk nilai kembalian.

1.3 Instruction Set (Set Instruksi)

Berikut adalah set instruksi *stack machine* yang digunakan pada kompilator Arion Milestone 4:

Instruksi	Format	Deskripsi
INT	INT L N	Alokasikan N slot memori pada blok L (0 = global)
LIT	LIT 0 V	<i>Push</i> literal V (angka/string/char/boolean) ke <i>stack</i>
LOD	LOD L A	<i>Push</i> nilai dari memori blok L offset A ke <i>stack</i>
STO	STO L A	<i>Pop</i> puncak <i>stack</i> , simpan ke memori blok L offset A
LDA	LDA L A	<i>Push</i> alamat (blok L, offset A) ke <i>stack</i>
LDI	LDI 0 0	<i>Load indirect: pop</i> alamat, <i>push</i> nilai di alamat tersebut
STI	STI 0 0	<i>Store indirect: pop</i> nilai dan alamat, simpan nilai ke alamat
OPR	OPR 0 K	Eksekusi operasi ke-K (aritmatika, relasional, I/O, dll.)
JMP	JMP 0 T	Lompat tanpa syarat ke instruksi alamat T
JPC	JPC 0 T	Lompat ke T jika puncak <i>stack</i> bernilai <i>false</i> (0)
CAL	CAL B T	Panggil subprogram di alamat T dengan blok B
RET	RET 0 0	Kembali dari subprogram; buang <i>frame</i> aktivasi
CHK	CHK L H	Periksa indeks array di puncak <i>stack</i> ada di rentang [L..H]
POP	POP 0 0	Buang nilai di puncak <i>stack</i>

Table 1: Set instruksi *stack machine* kompilator Arion

1.4 Kode Operasi (OPR)

Instruksi OPR meng-*encode* berbagai operasi melalui argumen kode operasi:

Kode	Operasi	Kode	Operasi
1	NEG (negasi unary)	12	GEQ ($\geq$)
2	ADD (+)	13	READLN (baca input)
3	SUB (−)	14	WRITE (cetak)
4	MUL ($\times$)	15	NOT (negasi logika)
5	DIV ($\div$ / <code>div</code>)	16	AND (<code>and</code>)
6	MOD (<code>mod</code>)	17	OR (<code>or</code>)
7	EQL (=)	18	NEWLINE (baris baru)
8	NEQ ($\neq$)	19	TO_CHAR (konversi ke <code>char</code>)
9	LSS (<)	20	NOOP (tanpa operasi)
10	LEQ ($\leq$)	21	RDIV (pembagian real)
11	GTR (>)		

Table 2: Kode operasi untuk instruksi OPR

Catatan: Kode operasi 1–14 mengikuti konvensi utama spesifikasi Milestone 4. Kode 15–20 merupakan tambahan untuk operasi boolean, *newline*, konversi `char`, dan *no-op*. Subbagian 2.2.

1.5 Mekanisme Pemanggilan Subprogram

Saat sebuah *procedure* atau *function* dipanggil melalui instruksi `CAL`, *interpreter* melakukan langkah-langkah berikut:

1. Membuat *activation frame* baru dengan `blockId` sesuai parameter instruksi.
2. Mengambil (*pop*) argumen aktual dari *operand stack* dan menyalinnya ke slot parameter dalam *frame*.
3. Menyimpan *return address* (nilai PC saat ini) ke dalam *frame*.
4. Memindahkan PC ke alamat instruksi pertama subprogram.

Saat instruksi `RET` ditemui, *frame* terakhir dibuang (*pop*) dari tumpukan *frame*. Jika subprogram adalah *function*, nilai di slot 0 *frame* (yang telah diisi oleh *assignment* ke nama fungsi) di-*push* ke *operand stack* sebagai nilai kembalian. PC dikembalikan ke *return address* yang tersimpan.

2 Perancangan dan Implementasi

2.1 Perancangan Program

Perancangan program pada Milestone 4 melanjutkan arsitektur *modular* berbasis OOP yang telah dibangun sejak Milestone 1. Dua komponen utama yang ditambahkan adalah:

- **Teknologi yang digunakan:** GNU C/C++ (standar C++17) dan Makefile.
- **Struktur Program & Komponen Baru:**
 - `IntermediateCode.hpp`: Mendefinisikan struktur data `Instruction` (satu baris instruksi IC dengan field `op`, `level`, `arg`, `literal`, `comment`), enum `OprCode` yang memetakan kode operasi ke konstanta bernama, dan alias `Code = vector<Instruction>`.
 - `IntermediateCodeGenerator (.hpp/.cpp)`: Kelas yang melakukan *traversal* pada *Decorated AST* secara *recursive descent* untuk menghasilkan rangkaian instruksi IC. Kelas ini bertanggung jawab atas: *memory layouting* (menentukan alamat setiap variabel global dan lokal), *code emission* (menambahkan instruksi ke vektor `code`), serta *backpatching* (mengisi target *jump* yang belum diketahui saat instruksi dibuat).
 - `StackInterpreter (.hpp/.cpp)`: Kelas yang mengeksekusi rangkaian instruksi IC menggunakan model *stack machine*. Kelas ini mengelola *global memory*, *activation frames*, dan *operand stack*. Setiap instruksi dieksekusi dalam siklus *fetch-decode-execute*.
 - `main.cpp`: Diperluas untuk mendukung Fase 4 — setelah *Semantic Analysis* berhasil, program memanggil `IntermediateCodeGenerator` untuk menghasilkan IC, mencetak IC ke terminal dan file, lalu menjalankan IC menggunakan `StackInterpreter` dan menyimpan *runtime output*.
- **Alur Kerja Program (Pipeline Lengkap):** Program beroperasi dalam empat fase berurutan:
 1. **Fase 1 — Lexical Analysis:** `ArionLexer` membaca file sumber dan menghasilkan daftar token.
 2. **Fase 2 — Syntax Analysis:** Parser membangun *parse tree*/AST dari daftar token.
 3. **Fase 3 — Semantic Analysis:** `SemanticAnalyzer` mendekorasi AST dengan informasi tipe, *scope*, dan *symbol table*.
 4. **Fase 4 — IC Generation & Interpretation:** `IntermediateCodeGenerator` mengubah *Decorated AST* menjadi instruksi IC, lalu `StackInterpreter` menjalankan instruksi IC tersebut dan menghasilkan output program.

Jika salah satu fase gagal (misalnya *syntax error* atau *semantic error*), fase-fase selanjutnya di-*skip* dan pesan *error* ditulis ke file output.
- **Justifikasi Implementasi:**
 - *Stack machine* dipilih sebagai model eksekusi karena pemetaan ekspresi aritmatika dari AST ke instruksi *stack* bersifat langsung dan natural — setiap

leaf node menghasilkan LIT/LOD dan setiap operator menghasilkan OPR.

- **Addressing** berbasis LDA/LDI/STI digunakan untuk mendukung akses elemen *array* dan *field record* secara seragam melalui komputasi alamat di *stack*.
- **Backpatching** digunakan untuk menangani *forward jumps* pada konstruksi IF, WHILE, FOR, dan CASE di mana target *jump* belum diketahui saat instruksi *jump* dibuat.
- **Pemisahan IC Generator dan Interpreter** ke dalam kelas terpisah memudahkan pengujian dan debugging masing-masing komponen secara independen.

2.2 Konvensi Tambahan di Luar Spesifikasi Dasar

Selama tahap perancangan dan implementasi, ditemukan kebutuhan untuk menambahkan beberapa instruksi dasar dan kode operasi (OPR) yang tidak tercakup dalam spesifikasi standar Milestone 4. Spesifikasi awal hanya mencakup 9 instruksi utama dan 14 kode operasi OPR. Penambahan ini dilakukan untuk mendukung fitur bahasa Arion yang lebih kompleks, seperti operasi *array*, akses memori tidak langsung (*pointers*/referensi), I/O baca, operasi logika boolean, serta komputasi pembagian *real*.

Berikut adalah daftar instruksi dan kode operasi tambahan serta modifikasi yang diimplementasikan di luar spesifikasi dasar:

1. Instruksi *Stack Machine* Tambahan

No	Instruksi	Deskripsi & Justifikasi
1	LDA L A	Load Address: <i>Push</i> alamat memori (blok L, offset A) ke puncak <i>stack</i> . Sangat krusial untuk kalkulasi alamat dinamis secara <i>runtime</i> , seperti saat mengakses indeks <i>array</i> atau manipulasi referensi <i>record</i> .
2	LDI 0 0	Load Indirect: Melakukan <i>pop</i> alamat dari <i>stack</i> , lalu melakukan <i>push</i> nilai yang tersimpan di alamat tersebut. Instruksi ini digunakan berpasangan dengan LDA untuk dereferensi alamat menjadi nilai.
3	STI 0 0	Store Indirect: Melakukan <i>pop</i> nilai dan alamat dari <i>stack</i> , kemudian menyimpan nilai tersebut secara langsung ke alamat target. Diperlukan untuk memfasilitasi operasi <i>assignment</i> ke elemen <i>array</i> .
4	CHK L H	Check Array Bounds: Mengevaluasi indeks array di puncak <i>stack</i> guna memastikan nilainya berada di antara rentang batas bawah (L) dan batas atas (H). Ini merupakan fitur pengaman terhadap kerentanan <i>Out-of-Bounds Variable Access</i> .
5	POP 0 0	Pop Value: Membuang nilai teratas dari <i>stack</i> . Fungsi utamanya adalah membersihkan tumpukan memori setelah pemanggilan prosedur/fungsi yang nilai kembaliannya tidak digunakan.

Table 3: Instruksi dasar tambahan di luar spesifikasi

2. Kode Operasi (OPR) Tambahan dan Modifikasi

Terdapat restrukturisasi pada *opcode* 13 dan 14, serta penambahan instruksi logika (boolean) dan *newline* sebagai kompensasi perubahan instruksi standar:

Kode	Mnemonic	Deskripsi & Justifikasi
13	OPR_READLN	[Modifikasi Spesifikasi] Pada spesifikasi adalah WRT. Dimodifikasi untuk memberikan fungsionalitas input (membaca ketikan pengguna secara <i>runtime</i>).
14	OPR_WRITE	[Modifikasi Spesifikasi] Pada spesifikasi adalah WRTLN. Dimodifikasi untuk sekadar mencetak teks ke layar tanpa secara otomatis membuat baris baru.
15	OPR_NOT	Logical NOT: Melakukan operasi negasi logika (<i>unary</i>) pada nilai boolean di puncak <i>stack</i> .
16	OPR_AND	Logical AND: Mengevaluasi konjungsi dari dua operand boolean di <i>stack</i> .
17	OPR_OR	Logical OR: Mengevaluasi disjungsi dari dua operand boolean di <i>stack</i> .
18	OPR_NEWLINE	Print Newline: Mencetak baris baru secara eksplisit. Instruksi ini ditambahkan sebagai kompensasi dihilangkannya WRTLN dari posisi <i>opcode</i> 14.
19	OPR_TO_CHAR	Convert to Char: Konversi <i>typecasting</i> secara <i>runtime</i> menjadi tipe data <i>char</i> .
20	OPR_NOOP	No Operation: Instruksi kosong untuk penahan tempat (<i>placeholder</i>) yang tidak memanipulasi <i>stack</i> sama sekali.
21	OPR_RDIV	Real Division (/). Membedakan pembagian nilai desimal (<i>real division</i>) secara ketat dengan pembagian bulat murni (<i>div</i> pada <i>opcode</i> 5). Mengembalikan nilai bertipe <i>real</i> dari 2 operand integer/real.

Table 4: Kode operasi (OPR) tambahan dan penyesuaian konvensi

2.3 Hasil Implementasi

Berikut adalah dokumentasi dari komponen-komponen penting dalam implementasi Milestone 4:

1. Struktur Data Instruksi dan Kode Operasi

Bagian ini mendefinisikan representasi satu baris instruksi IC beserta enum kode operasi yang digunakan oleh instruksi OPR.

Listing 1: Struktur data Instruction dan enum OprCode

```

struct Instruction {
    std::string op;           // mnemonic instruksi: LIT, LOD, STO,
    OPR, JMP, dst
    int level = 0;           // untuk global/frame/procedure block.
                                // Untuk CHK, dipakai sebagai low bound

```

```

    int arg = 0;           // operand angka/alamat/target jump.
                           // Untuk CHK, dipakai sebagai high bound
    std::string literal; // operand non-angka, misalnya string
                           literal
                           // atau metadata CAL
    std::string comment; // komentar yang ikut ditulis ke file IC
                           // setelah tanda ';'
};

enum OprCode {
    OPR_NEG = 1, OPR_ADD = 2, OPR_SUB = 3,
    OPR_MUL = 4, OPR_DIV = 5, OPR_MOD = 6,
    OPR_EQ = 7, OPR_NEQ = 8, OPR_LSS = 9,
    OPR_LEQ = 10, OPR_GTR = 11, OPR_GEQ = 12,
    OPR_READLN = 13, OPR_WRITE = 14,
    OPR_NOT = 15, OPR_AND = 16, OPR_OR = 17,
    OPR_NEWLINE = 18, OPR_TO_CHAR = 19, OPR_NOOP = 20,
    OPR_RDIV = 21 // real division
};

```

Penjelasan: Setiap instruksi terdiri dari *opcode* (*op*), level blok, argumen numerik, serta opsional literal dan comment. Khusus instruksi CHK, field *level* dan *arg* digunakan sebagai batas bawah dan batas atas indeks array. Enum *OprCode* memetakan setiap operasi ke konstanta bernama sehingga kode generator tidak bergantung pada angka mentah. Kode 1–14 mengikuti konvensi utama; kode 15–20 untuk boolean, I/O tambahan, dan *no-op*; kode 21 (*OPR_RDIV*) untuk pembagian real.

2. Entry Point IC Generator dan Memory Layouting

Fungsi `generate()` merupakan titik masuk utama yang mereset *state*, menyiapkan *layout* memori, lalu men-*generate* program utama.

Listing 2: Entry point dan memory layouting pada `IntermediateCodeGenerator`

```

Code IntermediateCodeGenerator::generate(ASTNode* root,
                                         const std::string& outFilename) {
    code.clear(); locations.clear();
    subprograms.clear(); subprogramOrder.clear();
    globalSize = 0; currentBlockId = 0;
    currentFunctionTabIdx = -1;
    if (!root) return code;
    prepareLayouts(root);
    genProgram(root);
    if (!outFilename.empty()) writeToFile(outFilename);
    return code;
}

```

```

}

void IntermediateCodeGenerator::prepareLayouts(ASTNode* root) {
    ASTNode* declPart = childNT(root, "<declaration-part>");
    collectGlobalVars(declPart);
    collectSubprograms(declPart);
    for (int idx : subprogramOrder)
        buildSubprogramLayout(subprograms[idx]);
}

```

Penjelasan: Sebelum satu pun instruksi di-*emit*, fungsi `prepareLayouts` mengumpulkan seluruh deklarasi variabel global dan subprogram dari *Decorated AST*. Variabel global ditempatkan di `blockId = 0` dengan *offset* naik sesuai ukuran tipe, sedangkan setiap subprogram memperoleh `blockId` tersendiri dengan *layout frame* yang mencakup slot parameter dan variabel lokal. Field `currentFunctionTabIdx` di-reset ke `-1` untuk melacak apakah kode sedang di-*generate* di dalam sebuah *function* (digunakan untuk menentukan alamat nilai kembalian fungsi pada slot 0 frame).

3. Backpatching pada Struktur Kontrol

Teknik *backpatching* digunakan untuk menangani instruksi lompat bersyarat (JPC) dan tak bersyarat (JMP) yang target alamatnya belum diketahui saat instruksi tersebut di-*emit*.

Listing 3: Backpatching pada `genIf` untuk konstruksi IF-THEN-ELSE

```

void IntermediateCodeGenerator::genIf(ASTNode* node) {
    // ... parsing children AST ...
    if (cond) genExpression(cond);
    int jpc = emit("JPC", 0, 0, "", "if false");
    if (thenStmt) genStatement(thenStmt);
    if (elseStmt) {
        int jmpEnd = emit("JMP", 0, 0, "", "skip else");
        patchArg(jpc, static_cast<int>(code.size()));
        genStatement(elseStmt);
        patchArg(jmpEnd, static_cast<int>(code.size()));
    } else {
        patchArg(jpc, static_cast<int>(code.size()));
    }
}

```

Penjelasan: Saat JPC di-*emit*, target lompat diisi sementara dengan 0. Setelah kode blok `then` selesai di-*generate*, alamat instruksi berikutnya (yaitu awal blok `else` atau akhir `if`) digunakan untuk mengisi kembali (*patch*) target lompat tersebut.

4. Fetch-Decode-Execute Cycle pada StackInterpreter

Inti dari *interpreter* adalah siklus *fetch-decode-execute* yang membaca instruksi berdasarkan PC, mendekodekan *opcode*, lalu mengeksekusinya.

Listing 4: Siklus eksekusi utama pada StackInterpreter::run

```
void StackInterpreter::run(std::ostream& out) {
    pc = 0; halted = false; frameDepth = 0;
    globalMemory.clear(); frames.clear(); operandStack.clear();
    while (!halted && pc >= 0 &&
           pc < static_cast<int>(program.size())) {
        const Instruction& ins = program[pc++];
        if (ins.op == "INT") {
            if (ins.arg < 0)
                throw std::runtime_error("negative memory size");
            if (ins.level == 0)
                globalMemory.assign(ins.arg, RuntimeValue::none());
            else if (!frames.empty() && ...)
                frames.back().mem.resize(ins.arg, ...);
        } else if (ins.op == "LIT") {
            push(parseLiteral(!ins.literal.empty() ?
                             ins.literal : std::to_string(ins.arg)));
        } else if (ins.op == "LDA") {
            push(RuntimeValue::address(ins.level, ins.arg));
        } else if (ins.op == "LDI") {
            RuntimeValue addr = pop();
            push(resolveAddress(addr));
        } else if (ins.op == "STI") {
            RuntimeValue val = pop();
            RuntimeValue addr = pop();
            resolveAddress(addr) = val;
        }
        // ... OPR, JMP, JPC, CAL, RET, CHK, POP ...
    }
}
```

Penjelasan: Setiap iterasi, instruksi diambil dari vektor `program` berdasarkan `pc` (yang langsung di-*increment*). Instruksi didekodekan berdasarkan *string* `op`-nya. `INT` mengalokasikan memori global atau me-*resize* frame lokal (dengan validasi ukuran non-negatif). `LIT` mem-*push* literal, `LDA` mem-*push* alamat, `LDI`/`STI` melakukan *load/store indirect* (masing-masing menyimpan alamat ke variabel sementara terlebih dahulu untuk menghindari masalah urutan evaluasi), dan `OPR` mendelegasikan ke `execOpr`. Field `frameDepth` melacak kedalaman pemanggilan subprogram untuk mendeteksi rekursi tak terbatas (`MAX_FRAME_DEPTH = 1000`).

3 Pengujian

Berikut adalah dokumentasi 10 pengujian mandiri dengan kasus yang unik untuk memastikan kebenaran *Intermediate Code Generation* dan *Stack Interpreter*. Setiap kasus uji menyertakan kode sumber Arion (*input*), Intermediate Code yang dihasilkan, serta *output* eksekusi program.

1. Kasus Uji 1: Assignment dan Aritmatika Dasar

Tujuan: Menguji alokasi memori global, *assignment* variabel, operasi penjumlahan, serta *output* `writeln`.

Input (Kode Sumber Arion):

```
program BasicAssign;
var x, y: integer;
begin
  x := 4;
  y := x + 6;
  writeln(y);
end.
```

```

File input      : test/milestone-4/input-1.txt
Output folder: test/milestone-4/output/input-1
Token output   : test/milestone-4/output/input-1/token_input-1.txt
Parse tree     : test/milestone-4/output/input-1/parse_tree_input-1.txt
Semantic AST   : test/milestone-4/output/input-1/decorated_ast_input-1.txt
IC output      : test/milestone-4/output/input-1/intermediate_code_input-1.txt
Run output     : test/milestone-4/output/input-1/run_input-1.txt

programsy
ident (BasicAssign)
semicolon

varsy
ident (x)
comma
ident (y)
colon
ident (integer)
semicolon

beginsy
ident (x)
becomes
intcon (4)
semicolon

ident (y)
becomes
ident (x)
plus
intcon (6)
semicolon

ident (writeln)
lparent
ident (y)
rparent
semicolon

endsy
period

```

Gambar 1: lexical analysis

```

=== Parse Tree ===
<program>
|-- <program-header>
|   |-- programsy(program)
|   |-- ident(BasicAssign)
|   |-- semicolon(;)
|-- <declaration-part>
|   |-- <var-declaration>
|       |-- varsy(var)
|       |-- <identifier-list>
|           |-- ident(x)
|           |-- comma(,)
|           |-- ident(y)
|       |-- colon(:)
|       |-- <type>
|           |-- ident(integer)
|       |-- semicolon(;)
|-- <compound-statement>
|   |-- beginsy(begin)
|   |-- <statement-list>
|       |-- <assignment-statement>
|           |-- ident(x)
|           |-- becomes(:=)
|           |-- <expression>
|               |-- <simple-expression>
|                   |-- <term>
|                       |-- <factor>
|                           |-- intcon(4)
|       |-- semicolon(;)
|       |-- <assignment-statement>
|           |-- ident(y)
|           |-- becomes(:=)
|           |-- <expression>
|               |-- <simple-expression>
|                   |-- <term>
|                       |-- <factor>
|                           |-- ident(x)
|                       |-- plus(+)
|                   |-- <term>
|                       |-- <factor>
|                           |-- intcon(6)
|       |-- semicolon(;)
|       |-- <procedure/function-call>
|           |-- ident(writeln)
|           |-- lparent((
|           |-- <parameter-list>
|               |-- <expression>
|                   |-- <simple-expression>
|                       |-- <term>
|                           |-- <factor>
|                               |-- ident(y)
|           |-- rparent())
|       |-- semicolon(;)
|-- endsy(end)
|-- period(.)

[SUCCESS] Parsing done. Tree saved to test/milestone-4\output\input-1\parse_tree_input-1.txt

```

Gambar 2: parse tree


```

=== Semantic Analysis ===

DECORATED AST:
-----
ProgramNode(name: 'BasicAssign', tab_index: 37)
  Declarations:
    VarDecl(name: 'x', type: integer, tab_index: 38, level: 0)
    VarDecl(name: 'y', type: integer, tab_index: 39, level: 0)
  Block(block_index: 1, lev: 1)
    Assign('x' := 4, type: void)
      Target:
        Var(name: 'x', tab_index: 38, type: integer)
      Value:
        Number(int: 4, type: integer)
    Assign('y' := x+6, type: void)
      Target:
        Var(name: 'y', tab_index: 39, type: integer)
      Value:
        BinOp(op: '+', type: integer)
          Left:
            Var(name: 'x', tab_index: 38, type: integer)
          Right:
            Number(int: 6, type: integer)
    ProcCall(name: 'writeln', tab_index: 35, type: void)
      Arguments:
        Var(name: 'y', tab_index: 39, type: integer)
-----

```

Gambar 3: semantic analysis

=== TAB (Identifier Table) ===								
Idx	Identifier	Link	Obj	Type	Ref	Nrm	Lev	Adr
0		0	TYPE	NOTYP	0	1	0	0
1	and	0	RESERVED	NOTYP	0	1	0	0
2	array	1	RESERVED	NOTYP	0	1	0	0
3	begin	2	RESERVED	NOTYP	0	1	0	0
4	case	3	RESERVED	NOTYP	0	1	0	0
5	const	4	RESERVED	NOTYP	0	1	0	0
6	div	5	RESERVED	NOTYP	0	1	0	0
7	downto	6	RESERVED	NOTYP	0	1	0	0
8	do	7	RESERVED	NOTYP	0	1	0	0
9	else	8	RESERVED	NOTYP	0	1	0	0
10	end	9	RESERVED	NOTYP	0	1	0	0
11	for	10	RESERVED	NOTYP	0	1	0	0
12	function	11	RESERVED	NOTYP	0	1	0	0
13	if	12	RESERVED	NOTYP	0	1	0	0
14	mod	13	RESERVED	NOTYP	0	1	0	0
15	not	14	RESERVED	NOTYP	0	1	0	0
16	of	15	RESERVED	NOTYP	0	1	0	0
17	or	16	RESERVED	NOTYP	0	1	0	0
18	procedure	17	RESERVED	NOTYP	0	1	0	0
19	program	18	RESERVED	NOTYP	0	1	0	0
20	record	19	RESERVED	NOTYP	0	1	0	0
21	repeat	20	RESERVED	NOTYP	0	1	0	0
22	integer	21	TYPE	INT	0	1	0	0
23	real	22	TYPE	REAL	0	1	0	0
24	boolean	23	TYPE	BOOL	0	1	0	0
25	char	24	TYPE	CHAR	0	1	0	0
26	string	25	TYPE	STR	0	1	0	0
27	then	26	RESERVED	NOTYP	0	1	0	0
28	to	27	RESERVED	NOTYP	0	1	0	0
29	type	28	RESERVED	NOTYP	0	1	0	0
30	until	29	RESERVED	NOTYP	0	1	0	0
31	var	30	RESERVED	NOTYP	0	1	0	0
32	while	31	RESERVED	NOTYP	0	1	0	0
33	true	32	CONST	BOOL	0	1	0	1
34	false	33	CONST	BOOL	0	1	0	0
35	writeln	34	PROC	NOTYP	0	1	0	0
36	readln	35	PROC	NOTYP	0	1	0	0
37	BasicAssign	36	PROGRAM	NOTYP	0	1	0	0
38	x	37	VAR	INT	0	1	0	0
39	y	38	VAR	INT	0	1	0	1

Gambar 4: identifier table

```

=== BTAB (Block Table) ===
Idx  Last  Lpar  Psze  Vsze
-----
  0     39     0     0     2
  1      0     0     0     0
-----

[Semantic analysis completed successfully]

[SUCCESS] Semantic analysis done. AST saved to test/milestone-4\output\input-1\decorated_ast_input-1.txt

=== Intermediate Code Generation ===
0 INT 0 2 ; allocate global memory
1 JMP 0 2 ; jump over subprogram bodies
2 LDA 0 0 ; address of x
3 LIT 0 4 ; literal
4 STI 0 0 ; store assignment target
5 LDA 0 1 ; address of y
6 LDA 0 0 ; address of x
7 LDI 0 0 ; load indirect
8 LIT 0 6 ; literal
9 OPR 0 2 ; simple-expression op
10 STI 0 0 ; store assignment target
11 LDA 0 1 ; address of y
12 LDI 0 0 ; load indirect
13 OPR 0 14 ; writeln arg
14 OPR 0 18 ; writeln newline
15 RET 0 0 ; end program

[SUCCESS] IC saved to test/milestone-4\output\input-1\intermediate_code_input-1.txt (16 instructions)

=== Program Output ===
10

[SUCCESS] Runtime output saved to test/milestone-4\output\input-1\run_input-1.txt

```

Gambar 5: BTAB, Intermediate Code Generation, Program output

2. Kasus Uji 2: Preseden Operator Aritmatika

Tujuan: Menguji urutan prioritas operator (*, div, +, -) dan penggunaan tanda kurung pada ekspresi.

Input (Kode Sumber Arion):

```

program ArithmeticPrecedence;
var a, b, c, d, z: integer;
begin
  a := 2; b := 3; c := 4; d := 8;
  z := (a + b) * c - d div 2;
  writeln(z);
end.

```

```
Run output : test/milestone-4/output/input-2/run_input-2.txt

programsy
ident (ArithmeticPrecedence)
semicolon

varsy
ident (a)
comma
ident (b)
comma
ident (c)
comma
ident (d)
comma
ident (z)
colon
ident (integer)
semicolon

beginsy
ident (a)
becomes
intcon (2)
semicolon

ident (b)
becomes
intcon (3)
semicolon

ident (c)
becomes
intcon (4)
semicolon

ident (d)
becomes
intcon (8)
semicolon

ident (z)
becomes
lparent
ident (a)
plus
ident (b)
rparent
times
ident (c)
minus
ident (d)
idiv
intcon (2)
semicolon

ident (writeln)
lparent
ident (z)
rparent
semicolon

endsy
period
```

Gambar 1: lexical analysis

```

=== Parse Tree ===
<program>
-- <program-header>
|-- programsy(program)
|-- ident(ArithmeticPrecedence)
|-- semicolon(;)
-- <declaration-part>
|-- <var-declaration>
|   |-- varsy(var)
|   |-- <identifier-list>
|       |-- ident(a)
|       |-- comma(,)
|       |-- ident(b)
|       |-- comma(,)
|       |-- ident(c)
|       |-- comma(,)
|       |-- ident(d)
|       |-- comma(,)
|       |-- ident(z)
|   |-- colon(:)
|   |-- <type>
|       |-- ident(integer)
|   |-- semicolon(;)
-- <compound-statement>
|-- beginsy(begin)
|-- <statement-list>
|   |-- <assignment-statement>
|       |-- ident(a)
|       |-- becomes(:=)
|       |-- <expression>
|           |-- <simple-expression>
|               |-- <term>
|                   |-- <factor>
|                       |-- intcon(2)
|       |-- semicolon(;)
|   |-- <assignment-statement>
|       |-- ident(b)
|       |-- becomes(:=)
|       |-- <expression>
|           |-- <simple-expression>
|               |-- <term>
|                   |-- <factor>
|                       |-- intcon(3)
|       |-- semicolon(;)
|   |-- <assignment-statement>
|       |-- ident(c)
|       |-- becomes(:=)
|       |-- <expression>
|           |-- <simple-expression>
|               |-- <term>
|                   |-- <factor>
|                       |-- intcon(4)
|       |-- semicolon(;)
|   |-- <assignment-statement>
|       |-- ident(d)
|       |-- becomes(:=)
|       |-- <expression>
|           |-- <simple-expression>
|               |-- <term>
|                   |-- <factor>
|                       |-- intcon(8)
|       |-- semicolon(;)
|   |-- <assignment-statement>
|       |-- ident(z)
|       |-- becomes(:=)
|       |-- <expression>

```

Gambar 2.1: parse tree

```

-- semicolon(;)
-- <assignment-statement>
|-- ident(z)
|-- becomes(:=)
|-- <expression>
|   |-- <simple-expression>
|       |-- <term>
|           |-- <factor>
|               |-- lpARENT((
|               |-- <expression>
|                   |-- <simple-expression>
|                       |-- <term>
|                           |-- <factor>
|                               |-- ident(a)
|                           |-- plus(+)
|                           |-- <term>
|                               |-- <factor>
|                                   |-- ident(b)
|               |-- rpARENT())
|           |-- times(*)
|           |-- <factor>
|               |-- ident(c)
|       |-- minus(-)
|       |-- <term>
|           |-- <factor>
|               |-- ident(d)
|           |-- idiv(div)
|           |-- <factor>
|               |-- intcon(2)
|   |-- semicolon(;)
|   |-- <procedure/function-call>
|       |-- ident(writeln)
|       |-- lpARENT((
|       |-- <parameter-list>
|           |-- <expression>
|               |-- <simple-expression>
|                   |-- <term>
|                       |-- <factor>
|                           |-- ident(z)
|       |-- rpARENT())
|   |-- semicolon(;)
|-- endsy(end)
|-- period(.)

[SUCCESS] Parsing done. Tree saved to test/milestone-4/output/input-2/parse_tree_input-2.txt

```

Gambar 2.2: parse tree

=== Semantic Analysis ===

DECORATED AST:

```

ProgramNode(name: 'ArithmeticPrecedence', tab_index: 37)
  Declarations:
    VarDecl(name: 'a', type: integer, tab_index: 38, level: 0)
    VarDecl(name: 'b', type: integer, tab_index: 39, level: 0)
    VarDecl(name: 'c', type: integer, tab_index: 40, level: 0)
    VarDecl(name: 'd', type: integer, tab_index: 41, level: 0)
    VarDecl(name: 'z', type: integer, tab_index: 42, level: 0)
  Block(block_index: 1, lev: 1)
    Assign('a' := 2, type: void)
      Target:
        Var(name: 'a', tab_index: 38, type: integer)
      Value:
        Number(int: 2, type: integer)
    Assign('b' := 3, type: void)
      Target:
        Var(name: 'b', tab_index: 39, type: integer)
      Value:
        Number(int: 3, type: integer)
    Assign('c' := 4, type: void)
      Target:
        Var(name: 'c', tab_index: 40, type: integer)
      Value:
        Number(int: 4, type: integer)
    Assign('d' := 8, type: void)
      Target:
        Var(name: 'd', tab_index: 41, type: integer)
      Value:
        Number(int: 8, type: integer)
    Assign('z' := lparent*c-ddiv2, type: void)
      Target:
        Var(name: 'z', tab_index: 42, type: integer)
      Value:
        BinOp(op: '-', type: integer)
          Left:
            BinOp(op: '*', type: integer)
              Left:
                BinOp(op: '+', type: integer)
                  Left:
                    Var(name: 'a', tab_index: 38, type: integer)
                  Right:
                    Var(name: 'b', tab_index: 39, type: integer)
                Right:
                    Var(name: 'c', tab_index: 40, type: integer)
              Right:
                BinOp(op: 'div', type: integer)
                  Left:
                    Var(name: 'd', tab_index: 41, type: integer)
                  Right:
                    Number(int: 2, type: integer)
          Right:
            ProcCall(name: 'writeln', tab_index: 35, type: void)
              Arguments:
                Var(name: 'z', tab_index: 42, type: integer)

```

Gambar 3: semantic analysis

=== TAB (Identifier Table) ===								
Idx	Identifier	Link	Obj	Type	Ref	Nrm	Lev	Adr
0		0	TYPE	NOTYP	0	1	0	0
1	and	0	RESERVED	NOTYP	0	1	0	0
2	array	1	RESERVED	NOTYP	0	1	0	0
3	begin	2	RESERVED	NOTYP	0	1	0	0
4	case	3	RESERVED	NOTYP	0	1	0	0
5	const	4	RESERVED	NOTYP	0	1	0	0
6	div	5	RESERVED	NOTYP	0	1	0	0
7	downto	6	RESERVED	NOTYP	0	1	0	0
8	do	7	RESERVED	NOTYP	0	1	0	0
9	else	8	RESERVED	NOTYP	0	1	0	0
10	end	9	RESERVED	NOTYP	0	1	0	0
11	for	10	RESERVED	NOTYP	0	1	0	0
12	function	11	RESERVED	NOTYP	0	1	0	0
13	if	12	RESERVED	NOTYP	0	1	0	0
14	mod	13	RESERVED	NOTYP	0	1	0	0
15	not	14	RESERVED	NOTYP	0	1	0	0
16	of	15	RESERVED	NOTYP	0	1	0	0
17	or	16	RESERVED	NOTYP	0	1	0	0
18	procedure	17	RESERVED	NOTYP	0	1	0	0
19	program	18	RESERVED	NOTYP	0	1	0	0
20	record	19	RESERVED	NOTYP	0	1	0	0
21	repeat	20	RESERVED	NOTYP	0	1	0	0
22	integer	21	TYPE	INT	0	1	0	0
23	real	22	TYPE	REAL	0	1	0	0
24	boolean	23	TYPE	BOOL	0	1	0	0
25	char	24	TYPE	CHAR	0	1	0	0
26	string	25	TYPE	STR	0	1	0	0
27	then	26	RESERVED	NOTYP	0	1	0	0
28	to	27	RESERVED	NOTYP	0	1	0	0
29	type	28	RESERVED	NOTYP	0	1	0	0
30	until	29	RESERVED	NOTYP	0	1	0	0
31	var	30	RESERVED	NOTYP	0	1	0	0
32	while	31	RESERVED	NOTYP	0	1	0	0
33	true	32	CONST	BOOL	0	1	0	1
34	false	33	CONST	BOOL	0	1	0	0
35	writeln	34	PROC	NOTYP	0	1	0	0
36	readln	35	PROC	NOTYP	0	1	0	0
37	ArithmeticPrecedence	36	PROGRAM	NOTYP	0	1	0	0
38	a	37	VAR	INT	0	1	0	0
39	b	38	VAR	INT	0	1	0	1
40	c	39	VAR	INT	0	1	0	2
41	d	40	VAR	INT	0	1	0	3
42	z	41	VAR	INT	0	1	0	4

Gambar 4: identifier table


```

=== BTAB (Block Table) ===
  Idx  Last  Lpar  Psze  Vsze
-----
    0    42     0     0     5
    1     0     0     0     0

[Semantic analysis completed successfully]

[SUCCESS] Semantic analysis done. AST saved to test/milestone-4/output/input-2/decorated_ast_input-2.txt

=== Intermediate Code Generation ===
0 INT 0 5 ; allocate global memory
1 JMP 0 2 ; jump over subprogram bodies
2 LDA 0 0 ; address of a
3 LIT 0 2 ; literal
4 STI 0 0 ; store assignment target
5 LDA 0 1 ; address of b
6 LIT 0 3 ; literal
7 STI 0 0 ; store assignment target
8 LDA 0 2 ; address of c
9 LIT 0 4 ; literal
10 STI 0 0 ; store assignment target
11 LDA 0 3 ; address of d
12 LIT 0 8 ; literal
13 STI 0 0 ; store assignment target
14 LDA 0 4 ; address of z
15 LDA 0 0 ; address of a
16 LDI 0 0 ; load indirect
17 LDA 0 1 ; address of b
18 LDI 0 0 ; load indirect
19 OPR 0 2 ; simple-expression op
20 LDA 0 2 ; address of c
21 LDI 0 0 ; load indirect
22 OPR 0 4 ; term op
23 LDA 0 3 ; address of d
24 LDI 0 0 ; load indirect
25 LIT 0 2 ; literal
26 OPR 0 5 ; term op
27 OPR 0 3 ; simple-expression op
28 STI 0 0 ; store assignment target
29 LDA 0 4 ; address of z
30 LDI 0 0 ; load indirect
31 OPR 0 14 ; writeln arg
32 OPR 0 18 ; writeln newline
33 RET 0 0 ; end program

[SUCCESS] IC saved to test/milestone-4/output/input-2/intermediate_code_input-2.txt (34 instructions)

=== Program Output ===
16

[SUCCESS] Runtime output saved to test/milestone-4/output/input-2/run_input-2.txt

```

Gambar 5: BTAB, Intermediate Code Generation, Program output

Output Eksekusi: 16

Penjelasan: $(2 + 3) \times 4 - 8 \div 2 = 20 - 4 = 16$. IC menghasilkan urutan yang benar: LIT operand, OPR ADD, OPR MUL, OPR DIV, OPR SUB.

3. Kasus Uji 3: Percabangan IF-THEN-ELSE (*else branch*)

Tujuan: Menguji instruksi JPC dan JMP pada konstruksi if-then-else di mana kondisi bernilai *false*.

Input (Kode Sumber Arion):

```

program IfElseFalse;
var x, y: integer;
begin
  x := 3;
  if x > 5 then y := 1 else y := 2;
  writeln(y);
end.

```

```

File input      : test/milestone-4/input-3.txt
Output folder:  test/milestone-4/output/input-3
Token output   : test/milestone-4/output/input-3/token_input-3.txt
Parse tree     : test/milestone-4/output/input-3/parse_tree_input-3.txt
Semantic AST   : test/milestone-4/output/input-3/decorated_ast_input-3.txt
IC output      : test/milestone-4/output/input-3/intermediate_code_input-3.txt
Run output     : test/milestone-4/output/input-3/run_input-3.txt

programsy
ident (IfElseFalse)
semicolon

varsy
ident (x)
comma
ident (y)
colon
ident (integer)
semicolon

beginsy
ident (x)
becomes
intcon (3)
semicolon

ifsy
ident (x)
gtr
intcon (5)
thensy
ident (y)
becomes
intcon (1)
elsesy
ident (y)
becomes
intcon (2)
semicolon

ident (writeln)
lparent
ident (y)
rparent
semicolon

endsy
period
=== Lexical Analysis Done ===

```

Gambar 1: lexical analysis

```

=== Parse Tree ===
<program>
|-- <program-header>
|  |-- programsy(program)
|  |-- ident(IfElseFalse)
|  |-- semicolon(;)
|  |-- <declaration-part>
|     |-- <var-declaration>
|        |-- varsy(var)
|           |-- <identifier-list>
|              |-- ident(x)
|              |-- comma(,)
|              |-- ident(y)
|              |-- colon(:)
|              |-- <type>
|                 |-- ident(integer)
|                 |-- semicolon(;)
|  |-- <compound-statement>
|     |-- beginsy(begin)
|     |-- <statement-list>
|        |-- <assignment-statement>
|           |-- ident(x)
|           |-- becomes(=)
|           |-- <expression>
|              |-- <simple-expression>
|                 |-- <term>
|                    |-- <factor>
|                       |-- intcon(3)
|           |-- semicolon(;)
|        |-- <if-statement>
|           |-- ifsy(if)
|           |-- <expression>
|              |-- <simple-expression>
|                 |-- <term>
|                    |-- <factor>
|                       |-- ident(x)
|           |-- gtr(>)
|           |-- <simple-expression>
|              |-- <term>
|                 |-- <factor>
|                    |-- intcon(5)
|           |-- thensy(then)
|           |-- <assignment-statement>
|              |-- ident(y)
|              |-- becomes(=)
|              |-- <expression>
|                 |-- <simple-expression>
|                    |-- <term>
|                       |-- <factor>
|                          |-- intcon(1)
|           |-- elsesy(else)
|           |-- <assignment-statement>
|              |-- ident(y)
|              |-- becomes(=)
|              |-- <expression>
|                 |-- <simple-expression>

```

Gambar 2.1: parse tree

```

|-- <else>(<else>)
|-- <assignment-statement>
|-- <ident(y)>
|-- <becomes(:)>
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- <intcon(2)>
|-- <semicolon(:)>
|-- <procedure/function-call>
|-- <ident(writeln)>
|-- <iparent(:)>
|-- <parameter-list>
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- <ident(y)>
|-- <rpresent(:)>
|-- <semicolon(:)>
|-- <endof(end)>
|-- <period(.)>
[SUCCESS] Parsing done. Tree saved to test/milestone-4/output/input-3/parse_tree.input-3.txt

```

Gambar 2.2: parse tree

```

=== Semantic Analysis ===

DECORATED AST:
-----
ProgramNode(name: 'IfElseFalse', tab_index: 37)
  Declarations:
    VarDecl(name: 'x', type: integer, tab_index: 38, level: 0)
    VarDecl(name: 'y', type: integer, tab_index: 39, level: 0)
  Block(block_index: 1, lev: 1)
    Assign('x' := 3, type: void)
      Target:
        Var(name: 'x', tab_index: 38, type: integer)
      Value:
        Number(int: 3, type: integer)
    If(type: void)
      Condition:
        BinOp(op: '>', type: boolean)
          Left:
            Var(name: 'x', tab_index: 38, type: integer)
          Right:
            Number(int: 5, type: integer)
      Then:
        Assign('y' := 1, type: void)
          Target:
            Var(name: 'y', tab_index: 39, type: integer)
          Value:
            Number(int: 1, type: integer)
      Else:
        Assign('y' := 2, type: void)
          Target:
            Var(name: 'y', tab_index: 39, type: integer)
          Value:
            Number(int: 2, type: integer)
    ProcCall(name: 'writeln', tab_index: 35, type: void)
      Arguments:
        Var(name: 'y', tab_index: 39, type: integer)
  -----

```

Gambar 3: semantic analysis

=== TAB (Identifier Table) ===								
Idx	Identifier	Link	Obj	Type	Ref	Nrm	Lev	Adr
0		0	TYPE	NOTYP	0	1	0	0
1	and	0	RESERVED	NOTYP	0	1	0	0
2	array	1	RESERVED	NOTYP	0	1	0	0
3	begin	2	RESERVED	NOTYP	0	1	0	0
4	case	3	RESERVED	NOTYP	0	1	0	0
5	const	4	RESERVED	NOTYP	0	1	0	0
6	div	5	RESERVED	NOTYP	0	1	0	0
7	downto	6	RESERVED	NOTYP	0	1	0	0
8	do	7	RESERVED	NOTYP	0	1	0	0
9	else	8	RESERVED	NOTYP	0	1	0	0
10	end	9	RESERVED	NOTYP	0	1	0	0
11	for	10	RESERVED	NOTYP	0	1	0	0
12	function	11	RESERVED	NOTYP	0	1	0	0
13	if	12	RESERVED	NOTYP	0	1	0	0
14	mod	13	RESERVED	NOTYP	0	1	0	0
15	not	14	RESERVED	NOTYP	0	1	0	0
16	of	15	RESERVED	NOTYP	0	1	0	0
17	or	16	RESERVED	NOTYP	0	1	0	0
18	procedure	17	RESERVED	NOTYP	0	1	0	0
19	program	18	RESERVED	NOTYP	0	1	0	0
20	record	19	RESERVED	NOTYP	0	1	0	0
21	repeat	20	RESERVED	NOTYP	0	1	0	0
22	integer	21	TYPE	INT	0	1	0	0
23	real	22	TYPE	REAL	0	1	0	0
24	boolean	23	TYPE	BOOL	0	1	0	0
25	char	24	TYPE	CHAR	0	1	0	0
26	string	25	TYPE	STR	0	1	0	0
27	then	26	RESERVED	NOTYP	0	1	0	0
28	to	27	RESERVED	NOTYP	0	1	0	0
29	type	28	RESERVED	NOTYP	0	1	0	0
30	until	29	RESERVED	NOTYP	0	1	0	0
31	var	30	RESERVED	NOTYP	0	1	0	0
32	while	31	RESERVED	NOTYP	0	1	0	0
33	true	32	CONST	BOOL	0	1	0	1
34	false	33	CONST	BOOL	0	1	0	0
35	writeln	34	PROC	NOTYP	0	1	0	0
36	readln	35	PROC	NOTYP	0	1	0	0
37	IfElseFalse	36	PROGRAM	NOTYP	0	1	0	0
38	x	37	VAR	INT	0	1	0	0
39	y	38	VAR	INT	0	1	0	1

Gambar 4: identifier table

```

=== BTAB (Block Table) ===
  Idx  Last  Lpar  Psze  Vsze
-----
    0    39     0     0     2
    1     0     0     0     0
-----

[Semantic analysis completed successfully]

[SUCCESS] Semantic analysis done. AST saved to test/milestone-4/output/input-3/decorated_ast_input-3.txt

=== Intermediate Code Generation ===
0 INT 0 2 ; allocate global memory
1 JMP 0 2 ; jump over subprogram bodies
2 LDA 0 0 ; address of x
3 LIT 0 3 ; literal
4 STI 0 0 ; store assignment target
5 LDA 0 0 ; address of x
6 LDI 0 0 ; load indirect
7 LIT 0 5 ; literal
8 OPR 0 11 ; relational op
9 JPC 0 14 ; if false
10 LDA 0 1 ; address of y
11 LIT 0 1 ; literal
12 STI 0 0 ; store assignment target
13 JMP 0 17 ; skip else
14 LDA 0 1 ; address of y
15 LIT 0 2 ; literal
16 STI 0 0 ; store assignment target
17 LDA 0 1 ; address of y
18 LDI 0 0 ; load indirect
19 OPR 0 14 ; writeln arg
20 OPR 0 18 ; writeln newline
21 RET 0 0 ; end program

[SUCCESS] IC saved to test/milestone-4/output/input-3/intermediate_code_input-3.txt (22 instructions)

=== Program Output ===
2

[SUCCESS] Runtime output saved to test/milestone-4/output/input-3/run_input-3.txt

```

Gambar 5: BTAB, Intermediate Code Generation, Program output

Output Eksekusi: 2

Penjelasan: Karena $x = 3$ tidak lebih besar dari 5, kondisi *false* menyebabkan JPC melompat melewati blok *then* ke blok *else*, sehingga $y = 2$.

4. Kasus Uji 4: Perulangan WHILE (Zero Iteration)

Tujuan: Menguji bahwa body *while* tidak dieksekusi jika kondisi awal sudah *false*.

Input (Kode Sumber Arion):

```

program WhileZeroIteration;
var x: integer;
begin
  x := 5;
  while x < 3 do begin x := x + 1; end;
  writeln(x);
end.

```

```

File input      : test/milestone-4/input-4.txt
Output folder   : test/milestone-4/output/input-4
Token output    : test/milestone-4/output/input-4/token_input-4.txt
Parse tree      : test/milestone-4/output/input-4/parse_tree_input-4.txt
Semantic AST    : test/milestone-4/output/input-4/decorated_ast_input-4.txt
IC output       : test/milestone-4/output/input-4/intermediate_code_input-4.txt
Run output      : test/milestone-4/output/input-4/run_input-4.txt

progransy
ident (WhileZeroIteration)
semicolon

varsy
ident (x)
colon
ident (integer)
semicolon

beginsy
ident (x)
becomes
intcon (5)
semicolon

whilesy
ident (x)
lss
intcon (3)
dosy
beginsy
ident (x)
becomes
ident (x)
plus
intcon (1)
semicolon

endsy
semicolon

ident (writeln)
lparent
ident (x)
rparent
semicolon

endsy
period
=== Lexical Analysis Done ===

```

Gambar 1: lexical analysis

```

=== Parse Tree ===
<program>
|-- <program-header>
|   |-- <programsy(program)>
|   |-- ident(WhileZeroIteration)
|   |-- semicolon(;)
|   |-- <declaration-part>
|       |-- <var-declaration>
|           |-- varsy(var)
|           |-- <identifier-list>
|               |-- ident(x)
|               |-- colon(:)
|               |-- <type>
|                   |-- ident(integer)
|                   |-- semicolon(;)
|       |-- <compound-statement>
|           |-- beginsy(begin)
|           |-- <statement-list>
|               |-- <assignment-statement>
|                   |-- ident(x)
|                   |-- becomes(=)
|                   |-- <expression>
|                       |-- <simple-expression>
|                           |-- <term>
|                               |-- <factor>
|                                   |-- intcon(5)
|                                   |-- intcon(5)
|                           |-- semicolon(;)
|                   |-- <while-statement>
|                       |-- <whilesy(while)>
|                           |-- <expression>
|                               |-- <simple-expression>
|                                   |-- <term>
|                                       |-- <factor>
|                                           |-- ident(x)
|                                           |-- lss(<)
|                                           |-- <simple-expression>
|                                               |-- <term>
|                                                   |-- <factor>
|                                                       |-- intcon(3)
|                                               |-- dosy(do)
|                                                   |-- <compound-statement>
|                                                       |-- beginsy(begin)
|                                                       |-- <statement-list>
|                                                           |-- <assignment-statement>
|                                                               |-- ident(x)
|                                                               |-- becomes(=)
|                                                               |-- <expression>
|                                                                   |-- <simple-expression>
|                                                                       |-- <term>
|                                                                           |-- <factor>
|                                                                               |-- ident(x)
|                                                                               |-- plus(+)
|                                                                               |-- <term>
|                                                                                   |-- <factor>
|                                                                                       |-- intcon(1)
|                                                           |-- semicolon(;)
|                                                       |-- endsy(end)
|                   |-- semicolon(;)
|           |-- <procedure/function-call>
|               |-- ident(writeln)
|               |-- lparent()
|               |-- <parameter-list>
|                   |-- <expression>
|                       |-- <simple-expression>
|                           |-- <term>
|                               |-- <factor>
|                                   |-- ident(x)
|               |-- rparent()
|               |-- semicolon(;)
|           |-- endsy(end)
|-- period(.)
[SUCCESS] Parsing done. Tree saved to test/milestone-4/output/input-4/parse_tree_input-4.txt

```

Gambar 2.1: parse tree

=== Semantic Analysis ===

DECORATED AST:

```
-----  
ProgramNode(name: 'WhileZeroIteration', tab_index: 37)  
  Declarations:  
    VarDecl(name: 'x', type: integer, tab_index: 38, level: 0)  
  Block(block_index: 1, lev: 1)  
    Assign('x' := 5, type: void)  
      Target:  
        Var(name: 'x', tab_index: 38, type: integer)  
      Value:  
        Number(int: 5, type: integer)  
    While(type: void)  
      Condition:  
        BinOp(op: '<', type: boolean)  
          Left:  
            Var(name: 'x', tab_index: 38, type: integer)  
          Right:  
            Number(int: 3, type: integer)  
      Block(block_index: 2, lev: 1)  
        Assign('x' := x+1, type: void)  
          Target:  
            Var(name: 'x', tab_index: 38, type: integer)  
          Value:  
            BinOp(op: '+', type: integer)  
              Left:  
                Var(name: 'x', tab_index: 38, type: integer)  
              Right:  
                Number(int: 1, type: integer)  
        ProcCall(name: 'writeln', tab_index: 35, type: void)  
          Arguments:  
            Var(name: 'x', tab_index: 38, type: integer)  
-----
```

Gambar 3: semantic analysis

=== TAB (Identifier Table) ===								
Idx	Identifier	Link	Obj	Type	Ref	Nrm	Lev	Adr
0		0	TYPE	NOTYP	0	1	0	0
1	and	0	RESERVED	NOTYP	0	1	0	0
2	array	1	RESERVED	NOTYP	0	1	0	0
3	begin	2	RESERVED	NOTYP	0	1	0	0
4	case	3	RESERVED	NOTYP	0	1	0	0
5	const	4	RESERVED	NOTYP	0	1	0	0
6	div	5	RESERVED	NOTYP	0	1	0	0
7	downto	6	RESERVED	NOTYP	0	1	0	0
8	do	7	RESERVED	NOTYP	0	1	0	0
9	else	8	RESERVED	NOTYP	0	1	0	0
10	end	9	RESERVED	NOTYP	0	1	0	0
11	for	10	RESERVED	NOTYP	0	1	0	0
12	function	11	RESERVED	NOTYP	0	1	0	0
13	if	12	RESERVED	NOTYP	0	1	0	0
14	mod	13	RESERVED	NOTYP	0	1	0	0
15	not	14	RESERVED	NOTYP	0	1	0	0
16	of	15	RESERVED	NOTYP	0	1	0	0
17	or	16	RESERVED	NOTYP	0	1	0	0
18	procedure	17	RESERVED	NOTYP	0	1	0	0
19	program	18	RESERVED	NOTYP	0	1	0	0
20	record	19	RESERVED	NOTYP	0	1	0	0
21	repeat	20	RESERVED	NOTYP	0	1	0	0
22	integer	21	TYPE	INT	0	1	0	0
23	real	22	TYPE	REAL	0	1	0	0
24	boolean	23	TYPE	BOOL	0	1	0	0
25	char	24	TYPE	CHAR	0	1	0	0
26	string	25	TYPE	STR	0	1	0	0
27	then	26	RESERVED	NOTYP	0	1	0	0
28	to	27	RESERVED	NOTYP	0	1	0	0
29	type	28	RESERVED	NOTYP	0	1	0	0
30	until	29	RESERVED	NOTYP	0	1	0	0
31	var	30	RESERVED	NOTYP	0	1	0	0
32	while	31	RESERVED	NOTYP	0	1	0	0
33	true	32	CONST	BOOL	0	1	0	1
34	false	33	CONST	BOOL	0	1	0	0
35	writeln	34	PROC	NOTYP	0	1	0	0
36	readln	35	PROC	NOTYP	0	1	0	0
37	WhileZeroIteration	36	PROGRAM	NOTYP	0	1	0	0
38	x	37	VAR	INT	0	1	0	0

Gambar 4: identifier table


```

=== BTAB (Block Table) ===
Idx  Last  Lpar  Psze  Vsze
-----
  0    38     0     0     1
  1     0     0     0     0
  2    38     0     0     0
-----

[Semantic analysis completed successfully]

[SUCCESS] Semantic analysis done. AST saved to test/milestone-4/output/input-4/decorated_ast_input-4.txt

=== Intermediate Code Generation ===
0 INT 0 1 ; allocate global memory
1 JMP 0 2 ; jump over subprogram bodies
2 LDA 0 0 ; address of x
3 LIT 0 5 ; literal
4 STI 0 0 ; store assignment target
5 LDA 0 0 ; address of x
6 LDI 0 0 ; load indirect
7 LIT 0 3 ; literal
8 OPR 0 9 ; relational op
9 JPC 0 17 ; while false
10 LDA 0 0 ; address of x
11 LDA 0 0 ; address of x
12 LDI 0 0 ; load indirect
13 LIT 0 1 ; literal
14 OPR 0 2 ; simple-expression op
15 STI 0 0 ; store assignment target
16 JMP 0 5 ; while repeat
17 LDA 0 0 ; address of x
18 LDI 0 0 ; load indirect
19 OPR 0 14 ; writeln arg
20 OPR 0 18 ; writeln newline
21 RET 0 0 ; end program

[SUCCESS] IC saved to test/milestone-4/output/input-4/intermediate_code_input-4.txt (22 instructions)

=== Program Output ===
5

[SUCCESS] Runtime output saved to test/milestone-4/output/input-4/run_input-4.txt

```

Gambar 5: BTAB, Intermediate Code Generation, Program output

Output Eksekusi: 5

Penjelasan: Kondisi $5 < 3$ langsung *false*, sehingga JPC melompat keluar loop dan x tetap 5.

5. Kasus Uji 5: Perulangan WHILE (Akumulasi)

Tujuan: Menguji loop while yang berjalan beberapa iterasi untuk mengakumulasi nilai.

Input (Kode Sumber Arion):

```

program WhileSum;
var x, sum: integer;
begin
  x := 0; sum := 0;
  while x < 4 do begin
    x := x + 1;
    sum := sum + x;
  end;
  writeln(sum);
end.

```

```

File input      : test/milestone-4/input-5.txt
Output folder  : test/milestone-4/output/input-5
Token output   : test/milestone-4/output/input-5/token_input-5.txt
Parse tree     : test/milestone-4/output/input-5/parse_tree_input-5.txt
Semantic AST   : test/milestone-4/output/input-5/decorated_ast_input-5.txt
IC output      : test/milestone-4/output/input-5/intermediate_code_input-5.txt
Run output     : test/milestone-4/output/input-5/run_input-5.txt

programsy
ident (WhileSum)
semicolon

varsy
ident (x)
comma
ident (sum)
colon
ident (integer)
semicolon

beginsy
ident (x)
becomes
intcon (0)
semicolon

ident (sum)
becomes
intcon (0)
semicolon

whilesy
ident (x)
lss
intcon (4)
dosy
beginsy
ident (x)
becomes
ident (x)
plus
intcon (1)
semicolon

ident (sum)
becomes
ident (sum)
plus
ident (x)
semicolon

endsy
semicolon

ident (writeln)
lparent
ident (sum)
rparent
semicolon

endsy
period
=== Lexical Analysis Done ===

```

Gambar 1: lexical analysis



Gambar 2.1: parse tree



Gambar 2.2: parse tree

```

=== Semantic Analysis ===

DECORATED AST:
-----
ProgramNode(name: 'WhileSum', tab_index: 37)
  Declarations:
    VarDecl(name: 'x', type: integer, tab_index: 38, level: 0)
    VarDecl(name: 'sum', type: integer, tab_index: 39, level: 0)
  Block(block_index: 1, lev: 1)
    Assign('x' := 0, type: void)
      Target:
        Var(name: 'x', tab_index: 38, type: integer)
      Value:
        Number(int: 0, type: integer)
    Assign('sum' := 0, type: void)
      Target:
        Var(name: 'sum', tab_index: 39, type: integer)
      Value:
        Number(int: 0, type: integer)
  While(type: void)
    Condition:
      BinOp(op: '<', type: boolean)
        Left:
          Var(name: 'x', tab_index: 38, type: integer)
        Right:
          Number(int: 4, type: integer)
    Block(block_index: 2, lev: 1)
      Assign('x' := x+1, type: void)
        Target:
          Var(name: 'x', tab_index: 38, type: integer)
        Value:
          BinOp(op: '+', type: integer)
            Left:
              Var(name: 'x', tab_index: 38, type: integer)
            Right:
              Number(int: 1, type: integer)
      Assign('sum' := sum+x, type: void)
        Target:
          Var(name: 'sum', tab_index: 39, type: integer)
        Value:
          BinOp(op: '+', type: integer)
            Left:
              Var(name: 'sum', tab_index: 39, type: integer)
            Right:
              Var(name: 'x', tab_index: 38, type: integer)
      ProcCall(name: 'writeln', tab_index: 35, type: void)
        Arguments:
          Var(name: 'sum', tab_index: 39, type: integer)
    -----

```

Gambar 3: semantic analysis

=== TAB (Identifier Table) ===								
Idx	Identifier	Link	Obj	Type	Ref	Nrm	Lev	Adr
0		0	TYPE	NOTYP	0	1	0	0
1	and	0	RESERVED	NOTYP	0	1	0	0
2	array	1	RESERVED	NOTYP	0	1	0	0
3	begin	2	RESERVED	NOTYP	0	1	0	0
4	case	3	RESERVED	NOTYP	0	1	0	0
5	const	4	RESERVED	NOTYP	0	1	0	0
6	div	5	RESERVED	NOTYP	0	1	0	0
7	downto	6	RESERVED	NOTYP	0	1	0	0
8	do	7	RESERVED	NOTYP	0	1	0	0
9	else	8	RESERVED	NOTYP	0	1	0	0
10	end	9	RESERVED	NOTYP	0	1	0	0
11	for	10	RESERVED	NOTYP	0	1	0	0
12	function	11	RESERVED	NOTYP	0	1	0	0
13	if	12	RESERVED	NOTYP	0	1	0	0
14	mod	13	RESERVED	NOTYP	0	1	0	0
15	not	14	RESERVED	NOTYP	0	1	0	0
16	of	15	RESERVED	NOTYP	0	1	0	0
17	or	16	RESERVED	NOTYP	0	1	0	0
18	procedure	17	RESERVED	NOTYP	0	1	0	0
19	program	18	RESERVED	NOTYP	0	1	0	0
20	record	19	RESERVED	NOTYP	0	1	0	0
21	repeat	20	RESERVED	NOTYP	0	1	0	0
22	integer	21	TYPE	INT	0	1	0	0
23	real	22	TYPE	REAL	0	1	0	0
24	boolean	23	TYPE	BOOL	0	1	0	0
25	char	24	TYPE	CHAR	0	1	0	0
26	string	25	TYPE	STR	0	1	0	0
27	then	26	RESERVED	NOTYP	0	1	0	0
28	to	27	RESERVED	NOTYP	0	1	0	0
29	type	28	RESERVED	NOTYP	0	1	0	0
30	until	29	RESERVED	NOTYP	0	1	0	0
31	var	30	RESERVED	NOTYP	0	1	0	0
32	while	31	RESERVED	NOTYP	0	1	0	0
33	true	32	CONST	BOOL	0	1	0	1
34	false	33	CONST	BOOL	0	1	0	0
35	writeln	34	PROC	NOTYP	0	1	0	0
36	readln	35	PROC	NOTYP	0	1	0	0
37	WhileSum	36	PROGRAM	NOTYP	0	1	0	0
38	x	37	VAR	INT	0	1	0	0
39	sum	38	VAR	INT	0	1	0	1

Gambar 4: identifier table

```

=== BTAB (Block Table) ===
Idx  Last  Lpar  Psze  Vsze
-----
0     39     0     0     2
1     0     0     0     0
2     39     0     0     0

[Semantic analysis completed successfully]

[SUCCESS] Semantic analysis done. AST saved to test/milestone-4/output/input-5/decorated_ast_input-5.txt

=== Intermediate Code Generation ===
0 INT 0 2 ; allocate global memory
1 JMP 0 2 ; jump over subprogram bodies
2 LDA 0 0 ; address of x
3 LIT 0 0 ; literal
4 STI 0 0 ; store assignment target
5 LDA 0 1 ; address of sum
6 LIT 0 0 ; literal
7 STI 0 0 ; store assignment target
8 LDA 0 0 ; address of x
9 LDI 0 0 ; load indirect
10 LIT 0 4 ; literal
11 OPR 0 9 ; relational op
12 JPC 0 27 ; while false
13 LDA 0 0 ; address of x
14 LDA 0 0 ; address of x
15 LDI 0 0 ; load indirect
16 LIT 0 1 ; literal
17 OPR 0 2 ; simple-expression op
18 STI 0 0 ; store assignment target
19 LDA 0 1 ; address of sum
20 LDA 0 1 ; address of sum
21 LDI 0 0 ; load indirect
22 LDA 0 0 ; address of x
23 LDI 0 0 ; load indirect
24 OPR 0 2 ; simple-expression op
25 STI 0 0 ; store assignment target
26 JMP 0 8 ; while repeat
27 LDA 0 1 ; address of sum
28 LDI 0 0 ; load indirect
29 OPR 0 14 ; writeln arg
30 OPR 0 18 ; writeln newline
31 RET 0 0 ; end program

[SUCCESS] IC saved to test/milestone-4/output/input-5/intermediate_code_input-5.txt (32 instructions)

=== Program Output ===
10

[SUCCESS] Runtime output saved to test/milestone-4/output/input-5/run_input-5.txt

```

Gambar 5: BTAB, Intermediate Code Generation, Program output

Output Eksekusi: 10

Penjelasan: Loop berjalan 4 kali ($x = 1, 2, 3, 4$), sehingga $sum = 1 + 2 + 3 + 4 = 10$.

Pola IC: JPC untuk keluar loop dan JMP untuk kembali ke pengecekan kondisi.

4 Kesimpulan dan Saran

4.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa program *Intermediate Code Generator* dan *Stack Interpreter* untuk bahasa pemrograman “Arion” telah berhasil dibangun dan berfungsi sesuai spesifikasi. Kedua komponen ini melengkapi *pipeline* kompilator Arion menjadi empat tahap utuh: *Lexical Analysis*, *Syntax Analysis*, *Semantic Analysis*, serta *Intermediate Code Generation* & *Interpretation*.

Intermediate Code Generator berhasil mengubah *Decorated AST* menjadi rangkaian in-

struksi *stack machine* yang mencakup seluruh konstruksi bahasa Arion, termasuk: *assignment*, ekspresi aritmatika dan logika, percabangan (*if-then-else*, *case*), perulangan (*while*, *repeat-until*, *for-to/downto*), pemanggilan *procedure* dan *function* dengan parameter, serta akses elemen *array*. Teknik *backpatching* digunakan secara efektif untuk menangani *forward jump*.

Stack Interpreter berhasil mengeksekusi instruksi IC dengan benar melalui siklus *fetch-decode-execute*, mengelola memori melalui *global memory* dan *activation frames*, serta mendukung operasi I/O (*writeln*, *readln*). Hasil pengujian dari 38 kasus uji (termasuk 10 kasus uji representatif yang didokumentasikan) membuktikan bahwa output eksekusi program Arion konsisten dan sesuai dengan yang diharapkan.

4.2 Saran

Untuk pengembangan lebih lanjut, berikut adalah beberapa saran penyempurnaan:

1. **Optimasi Intermediate Code:** Menambahkan tahap optimasi IC seperti *constant folding* (evaluasi ekspresi konstanta saat kompilasi), *dead code elimination* (penghapusan kode yang tidak terjangkau), dan *peephole optimization* (penggabungan instruksi berurutan yang redundan, misalnya LDA yang langsung diikuti LDI menjadi LOD).
2. **Dukungan Nested Procedure/Function:** Saat ini *interpreter* menggunakan pencarian linear pada tumpukan *frame* untuk menemukan *frame* dengan *blockId* tertentu. Untuk mendukung akses variabel pada *scope* yang lebih dalam secara efisien, dapat ditambahkan *static link* atau *display register*.
3. **Code Generation ke Target Riil:** Menambahkan *back-end* yang mengubah IC menjadi kode mesin riil (misalnya x86-64 atau LLVM IR) sehingga program Arion dapat dikompilasi menjadi *executable* asli yang tidak membutuhkan *interpreter*.
4. **Dukungan Input Interaktif yang Lebih Baik:** Instruksi *readln* saat ini hanya membaca satu token dari *stdin*. Dapat dikembangkan agar mendukung pembacaan berbasis baris (*line-buffered*) dan konversi tipe otomatis sesuai deklarasi variabel target.

5 Lampiran

- **Link Repository GitHub:**
<https://github.com/Zannn210/WOK-Tubes-IF2224-2026>
- **Tabel Pembagian Tugas:**

Nama Lengkap	NIM	Tugas	Kontribusi
Fauzan Mohamad Abdul Ghani	13524113	Stack Interpreter: Mengimplementasikan seluruh kelas <code>StackInterpreter</code> termasuk <code>RuntimeValue</code> , siklus <i>fetch-decode-execute</i> pada <code>run()</code> , operasi aritmatika (<code>numericOp</code>), operasi perbandingan (<code>compareOp</code>), serta manajemen <i>activation frame</i> pada <code>execCal</code> .	20%
Muhammad Daffa Arrizki Yanma	13524133	IC Generator — Statement & Control Flow: Mengimplementasikan fungsi <code>genIf</code> , <code>genCase</code> , <code>genWhile</code> , <code>genRepeat</code> , <code>genFor</code> , dan <code>genAssignment</code> pada <code>IntermediateCodeGenerator.cpp</code> , termasuk logika <i>backpatching</i> untuk semua konstruksi kontrol.	20%
Eduard Daniel Ari-ajaya	13524129	IC Generator — Expression & Memory Layout: Mengimplementasikan <code>genExpression</code> , <code>genSimpleExpression</code> , <code>genTerm</code> , <code>genFactor</code> , dan <code>genConstant</code> . Merancang <i>memory layouting</i> (<code>prepareLayouts</code> , <code>collectGlobalVars</code> , <code>buildSubprogramLayout</code>) serta <i>variable addressing</i> (<code>genAddress</code> , <code>genVariableValue</code>).	20%
Safira Berlianti	13524128	Pipeline Integration: Merancang struktur data <code>Instruction</code> dan enum <code>OprCode</code> pada <code>IntermediateCode.hpp</code> . Mengintegrasikan Fase 4 ke dalam <code>main.cpp</code> sehingga pipeline berjalan end-to-end dari lexer hingga interpreter. Menambahkan penanganan error antar-fase (skip file output jika fase sebelumnya gagal).	20%
Reysha Syafitri MR	13524137	IC Generator — Subprogram & Testing: Mengimplementasikan <code>genProcFuncCall</code> dan <code>genSubprogram</code> untuk pemanggilan procedure/function. Membuat 38 kasus uji komprehensif, menyusun laporan PDF Milestone 4, serta mempersiapkan release GitHub.	20%

Table 5: Pembagian tugas anggota tim

6 Referensi

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd Edition). Pearson.
2. Hopcroft, J. E., Motwani, R., & Ullman, J. D. *Introduction to Automata Theory, Languages, and Computation* (3rd Edition). Pearson.
3. *Stack Machine* — *Wikipedia*. Diakses dari: https://en.wikipedia.org/wiki/Stack_machine
4. *Intermediate Representation* — *Wikipedia*. Diakses dari: https://en.wikipedia.org/wiki/Intermediate_representation
5. Wirth, N. (1996). *Compiler Construction*. Addison-Wesley.
6. *Guidebook Konvensi Intermediate Code* — *Spesifikasi Milestone 4 IF2224 TBFO*. Lampiran Spesifikasi Tugas Besar.